Real-Time HAT for Raspberry PI

Use Case Description

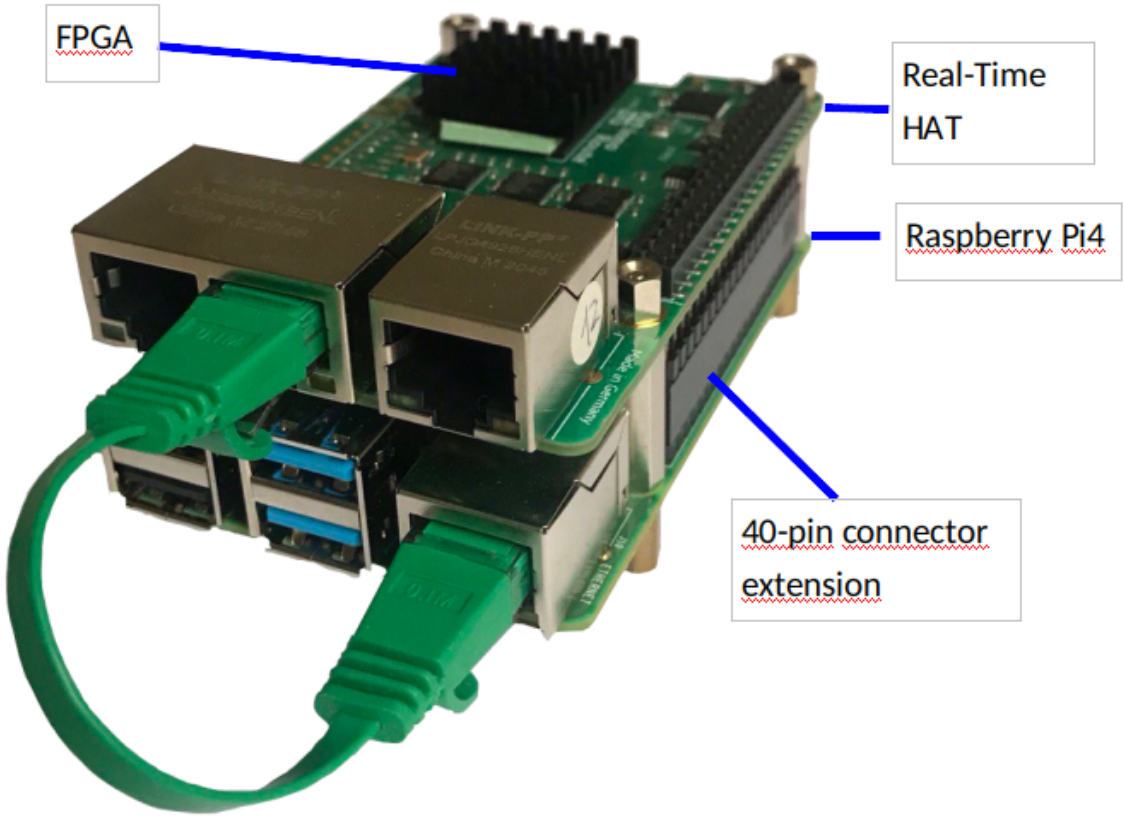

Table of Contents

Use Case Description 1
1 Introduction 2
2 Background 2
3 Use Case 0 3
4 Use Case 1 - Time Aware Scheduler (TAS) 3
4.1 Flow Cache 4
4.2 Time Aware Scheduler (TAS) 6
5 Test of Use Case 1 7
5.1 TSN-Analyzer 7
6 Use Case 4 11
7 Literature 11

1 Introduction

This document describes additional Use Cases for InnoRoute's Real-Time HAT (RTH) which can be activated by purchasing respective licenses. It sets upon the basic document "SystemDescriptionV2.pdf" from January 4, 2021. It can be downloaded here[1].

Use Case	Description
0	PTP Hardware time stamping (basic mode)
1	Time Aware Shaper (TAS) according to 802.1Qbv for up to 8 flows
4	Gigabit TAP

Table 1: List of currently available Use Cases.

2 Background

The RTH is a printed circuit board developed by InnoRoute as a Raspberry Pi conforming extension board (HAT = Hardware Attached on Top). The title figure above shows the RTH mounted on top of a Raspberry Pi4 (RP4). The RTH is controlled and powered via a 40-pin extension connector from the Raspberry Pi4. The user data connection is done via the (green) Ethernet cable from the Gigabit Ethernet port of the RP4 to the middle port (Port 1) of the 3 Gigabit Ethernet ports of the RTH.

The RTH has been designed from the beginning as an extensible platform. Its main processing element is a programmable microchip (FPGA = field programmable gate array). This device can be loaded with different functionality via a **bitstream**. Without bitstream the FPGA has no functionality.

The basic functionality of the RTH is precise time protocol (PTP) support. It allows to adjust the time base to an external clock master within a nanosecond range. See paper "Precise fruits: Hardware supported time-synchronisation on the RaspberryPI.", Ulbricht, Marian et al. 2021 International Conference on Smart Applications, Communications and Networking (SmartNets). IEEE, 2021[2].

PTP is the basis for further Time Sensitive Networking (TSN) protocols such as Time Aware Scheduler (TAS). TSN is an Ethernet technology specified in IEEE 802.1Q.

Images for the Pi4 are provided by InnoRoute, including the latest Raspberry Pi Operating System Bookworm-32, available here[3].

Lot of practical information is publicly available in our public GitHub[4]

The TSN functions of the RTH were successfully tested within interoperability test events (plugfests at LNI[5] and IIC[6]). Further interoperability testing with professional test equipment such as the TSN-Analyzer from eks Engel is described below.

[1] https://innoroute.com/download/systemdescription/
[2] https://ieeexplore.ieee.org/document/9555415
[3] https://my.hidrive.com/#$/users/innoroute/RTH_image_git_publish
[4] https://github.com/InnoRoute/RealtimeHAT/
[5] https://lni40.de/
[6] https://www.iiconsortium.org/

3 Use Case 0

The data flow in the basic operation mode is shown in Figure 1. All incoming packets from Ethernet ports RT0 (left RJ45 jack)) and RT1 (right RJ45 jack) are forwarded to the Ethernet port eth0 (middle RJ45 jack) which is connected to the Pi4 Ethernet port (see title page).

Further packet processing depends on the functionality implemented in the Pi4.

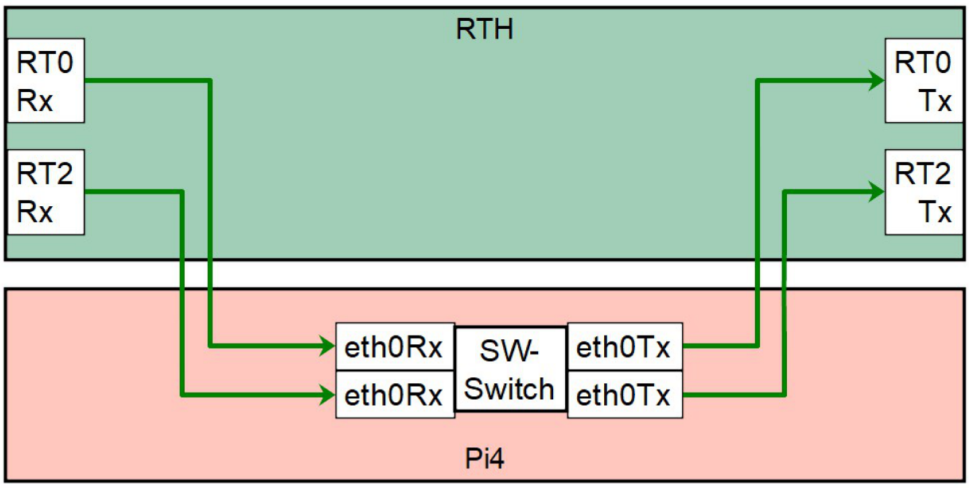

Figure 1: RTH Packet flows in Use Case 0

The nomenclature in Figure 1 and following figures uses the Linux port denomination, as it shows up when entering the command '**`ifconfig`**' in the command line interface (CLI).

4 Use Case 1 - Time Aware Scheduler (TAS)

This application works in cooperation with the Flow Cache. It allows to select up to eight flows for special treatment. The Flow Cache is common to RT0 and RT2 input ports and includes an Action Table to specify the destination of for the matching flows. In addition, a Time Aware Scheduler (TAS) is provided at each output port as shown in Figure 2. Packets which match one of the eight Flow Cache entries are directly forwarded to the specified output and priority queue.

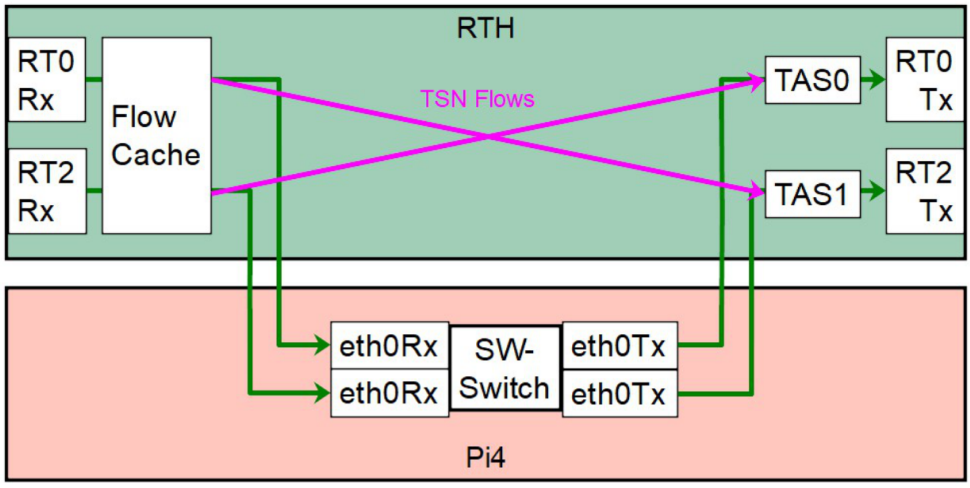

Figure 2: RTH Packet flows in Use Case 1

4.1 Flow Cache

The Flow Cache concept is summarized in Figure 3. For every packet from RT0 and RT2 inputs, VLAN-ID and PCP (Priority Code Point) are extracted and compared to eight preconfigured entries in the Match Table. In case of match in one row the action defined in the corresponding Action Table entry is executed. The action is direct forwarding to a defined output and a defined output Queue. The output is specified by the Pointer (Ptr.) to the Action Type Table, where three different forwarding types are predefined.

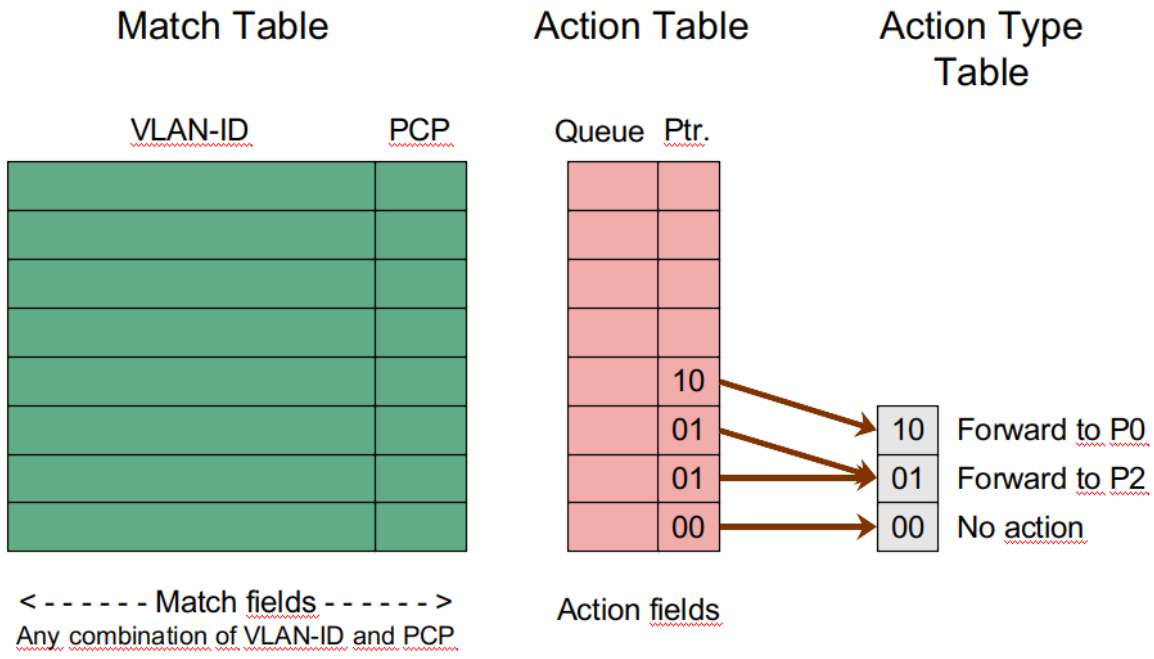

Figure 3: Flow Cache concept with exemplary Action Table configuration

The Flow Cache is configured with a handful of register write commands.

First, the Forwarding Type registers are configured:

Forwarding Type	Type	Address	Data
Default, forward to Pi4	0	0x00890000	0x00000000
Forward directly to RT2	2	0x00890004	0x00000011
Forward directly to RT0	1	0x00890008	0x00000001

Table 2: Configuration of Flow Type entries

Write to a register is done at command line interface (CLI) with the command:

```
/usr/share/InnoRoute/INR_mmi <Address> <Write_Data>
```

Note that only register write is supported, not the readout of registers.

Several write commands can be grouped for batch execution as for example the configuration of the Flow Type Table (Table 2):

```
/usr/share/InnoRoute/INR_mmi 0x00890000 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00890004 0x00000011
/usr/share/InnoRoute/INR_mmi 0x00890008 0x00000001
```

One Match and Action Table entry consists of nine registers, starting from a base address. The eight base addresses and the eight subsequent addresses are given in Table 3:

Entry	Base Address	Subsequent Addresses (last 3 digits)
#8	0x00c00-1c0	-1c4, -1c8, -1cc, -1d0, -1d4, -1d8, -1dc, -1e0
#7	0x00c00-180	-184, -188, -18c, -190, -194, -198, -19c, 1a0
#6	0x00c00-140	-144, -148, -14c, -150, -154, -158, -15c, -160
#5	0x00c00-100	-104, -108, -10c, -110, -114, -118, -11c, -120
#4	0x00c00-0c0	-0c4, -0c8, -0cc, -0d0, -0d4, -0d8, -0dc, -0e0
#3	0x00c00-080	-084, -088, -08c, -090, -094, -098, -09c, -0a0
#2	0x00c00-040	-044, -048, -04c, -050, -054, -058, -05c, -060
#1	0x00c00-000	-004, -008, -00c, -010, -014, -018, -01c, -020

Table 3: Flow Cache entries

The base register and the first subsequent register specify Match and Action Table entries as shown in Table 4 in green and red fields, respectively:

Register		Digits of Write Data							
Base	0x	0	0	PCP	Type	F	F	0	9
Base+1	0x	0	0	Queue	VLAN-ID			0	0
Base+2	0x	0	0	0	0	0	0	0	0
:	:	:	:	:	:	:	:	:	:
Base+8	0x	0	0	0	0	0	0	0	0
With:	Match fields (green): PCP and VLAN-ID Action fields (red): output Queue, Type=0, 1, 3 (see Table 2)								

Table 4: Flow Cache entry values

For example, to configure Entry #1 of the Flow Cache to forward packets with VLAN-ID=4 and PCP=3 directly to Port 2 via output Queue 3 the following command sequence applies:

```
/usr/share/InnoRoute/INR_mmi 0x00c00000 0x0033ff09
/usr/share/InnoRoute/INR_mmi 0x00c00004 0x00300400
/usr/share/InnoRoute/INR_mmi 0x00c00008 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00c0000c 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00c00010 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00c00014 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00c00018 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00c0001c 0x00000000
/usr/share/InnoRoute/INR_mmi 0x00c00020 0x00000000
```

Note that all nine register writes must be done in a sequence, although the values are zero; this is to complete the hardware write procedure. Accordingly, to clear one entry all registers must be written to zero in one sequence.

4.2 Time Aware Scheduler (TAS)

The goal of the TAS function is to achieve a timeslot behavior on a transmission line. Packet flows with periodic behavior are supported by periodic time slots which are exclusively reserved for them.

An example is shown in Figure 4 at the right side: the TSN slot within the periodic 10 ms cycle is reserved for a periodic packet flow. To make sure that no pending packets are left over from the preceding non-TSN slot a guard interval is accommodated, where no packets are admitted. This time is calculated using the longest Ethernet frame of 1500 byte and the transmission speed of 100 Mbit/s, which leads to about 0.12 ms. Within the periodic TSN slot of 1 ms size a maximum data rate of 10 Mb/s can be transported.

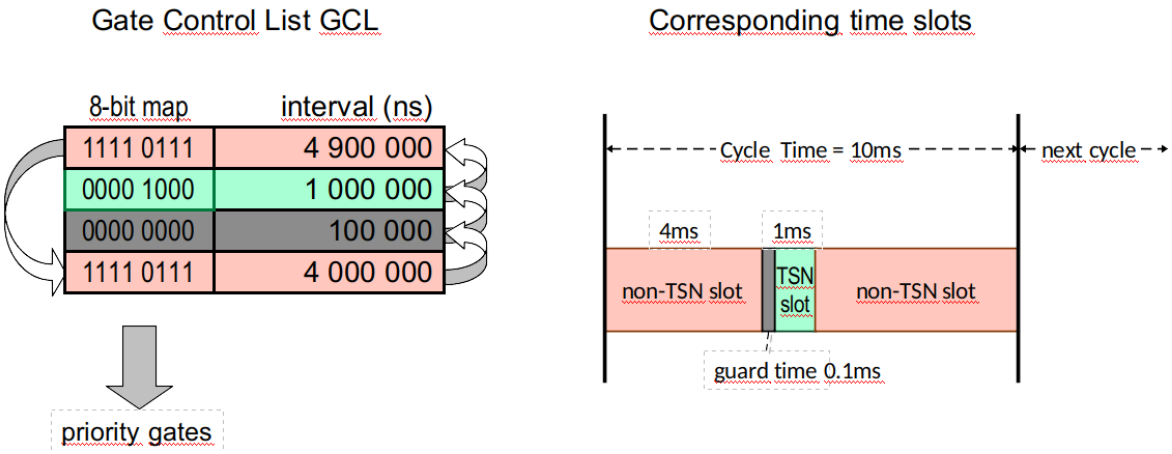

Figure 4: TAS concept with exemplary time slots

The realization of this behavior is done by the Gate Control List (GCL) shown at the left side of Figure 4. The 8-bit map controls opening (1) and closing (0) of the eight priority gates. The Flow Cache described in the previous section makes sure that only TSN packets are forwarded to the output Queue 3. The four entries in the GCL are worked off periodically and without gap by the hardware. Each entry consists of

- the duration of the period in nanoseconds, encoded as 32-bit value and
- the gate open/close information for the priority queues.

Configuration of the GCL is done by the following Python program:

```
# programm reads current fpga status
from rt_hat import TAS as RT_HAT_TAS
RT_HAT_TAS.DEBUG_ENABLE=True
RT_HAT_TAS.DEBUG_OUTPUT=True
RT_HAT_TAS.init("/usr/share/InnoRoute/hat_env.sh")
port=1
my_GCL=[
        [0xF7, 49000000], # all gates open except TSN (gate 3) for 4.9ms
        [0x00, 10000000], # all gates closed for 1ms > guard time
        [0x08,  1000000], # gate 3 open for 0.1ms >  TSN Traffic
        [0xF7, 40000000]  # non-TSN traffic for 4ms
        ]
RT_HAT_TAS.set_GCL(my_GCL,port) # set GCL of port
```

```
print(RT_HAT_TAS.get_GCL(port)) # print GCL
RT_HAT_TAS.apply(port)          # apply tas settings to port
```

Code: TAS configuration for PCP=3 with guard time slot

Note that '**port=1**' in the above code refers to RT2, while '**port=0**' would refer to RT0.

The Python program is executed with the CLI command '**python3 <program name>**'.

Within this program the yellow highlighted parts can be modified by the user to the desired output port and to the desired GCL with a maximum of 128 entries.

The Python program also checks the GCL parameter values for consistency and eventually adjusts the cycle time.

5 Test of Use Case 1

This chapter describes a test scenario to evaluate Flow Cache and TAS for a specific scenario. In this example the TSN-Analyzer from the company EKS Engel is used to generate and analyze packet streams. This device is described first in Section 4.1, then the TAS test in Section 4.2. Alternatively, other TSN measurement equipment can be used as well.

5.1 TSN-Analyzer

While the RTH will remain a development tool for laboratories based on command line interface (CLI) a commercial customer equipment has been produced based on the RTH plus RP4 combo (see title page figure). This device is exclusively operated with a Graphical User Interface (GUI) from the company Realtime-IT[7], has an industry-proven case, certifications and fulfills EMC requirements.

The device is manufactured and distributed by the partner EKS Engel[8].

Figure 5: TSN-Analyzer

The TSN-Analyzer initially supports these basic features:

1. PTP with master and slave option
2. Packet stream generation at Port P1 (left) and analysis at Port P2.
3. Semi-passive TAP with proprietary PTP and packet analysis.

[7] https://www.realtime-it.de/en/

[8] https://www.eks-engel.de/en/products/industrial-network-technology/the-industrial-ethernet/the-tsn-analyzer/

Packet stream generation is configured via the page shown in Figure 6. In this example eight streams with different priority and different VLAN-ID are specified. Packet size and insertion period are selectable as well.

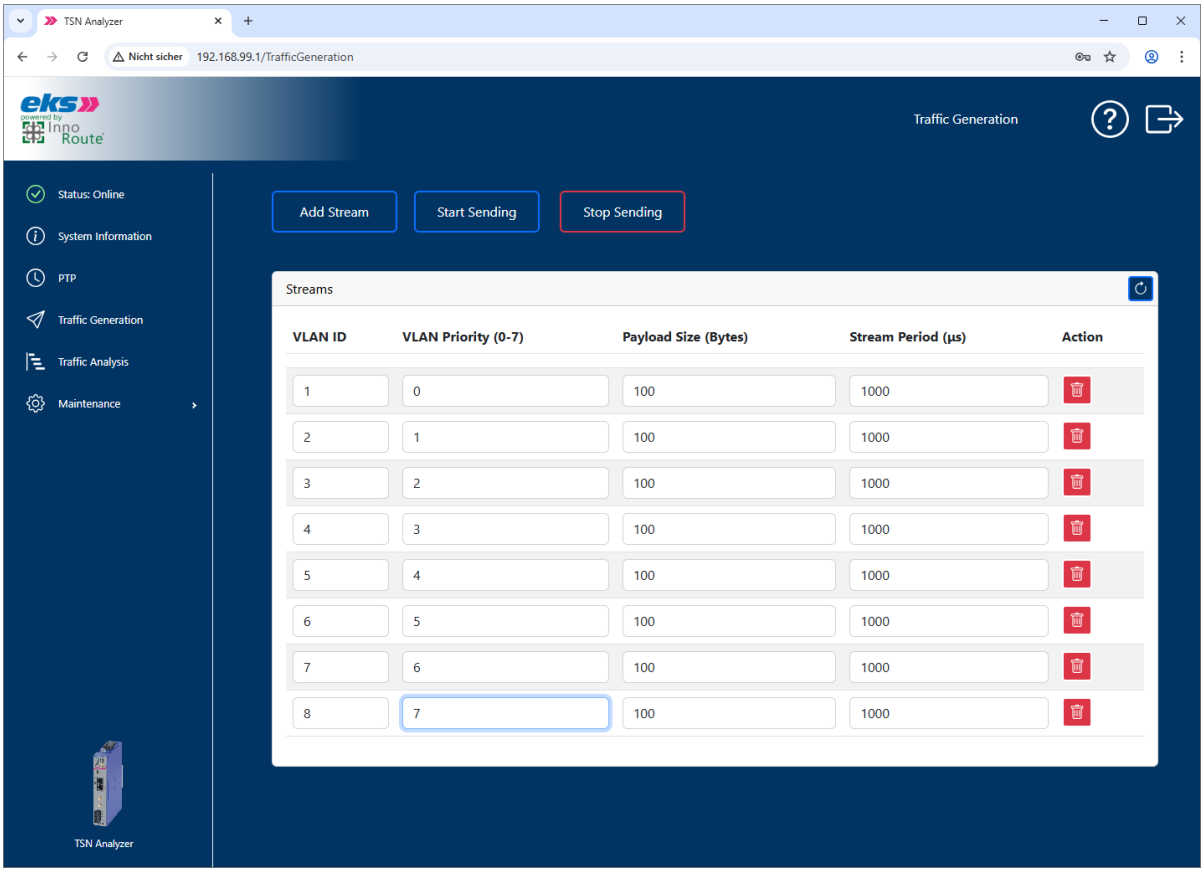

Figure 6: Packet streams generated with the TSN-Analyzer

The streams are generated by the Pi4 in software. Hence, the stream period is not exactly fulfilled. There will be jitter and also missing packets, especially if many streams are generated in parallel.

To test this configuration port P1 of the TSN-Analyzer is connected directly to the port P2 with a short, green cable as shown in Figure 7.

The TSN-Analyzer screen “Traffic Analysis” now shows in the lower part the stochastic packet arrival of all packet streams at port P2.

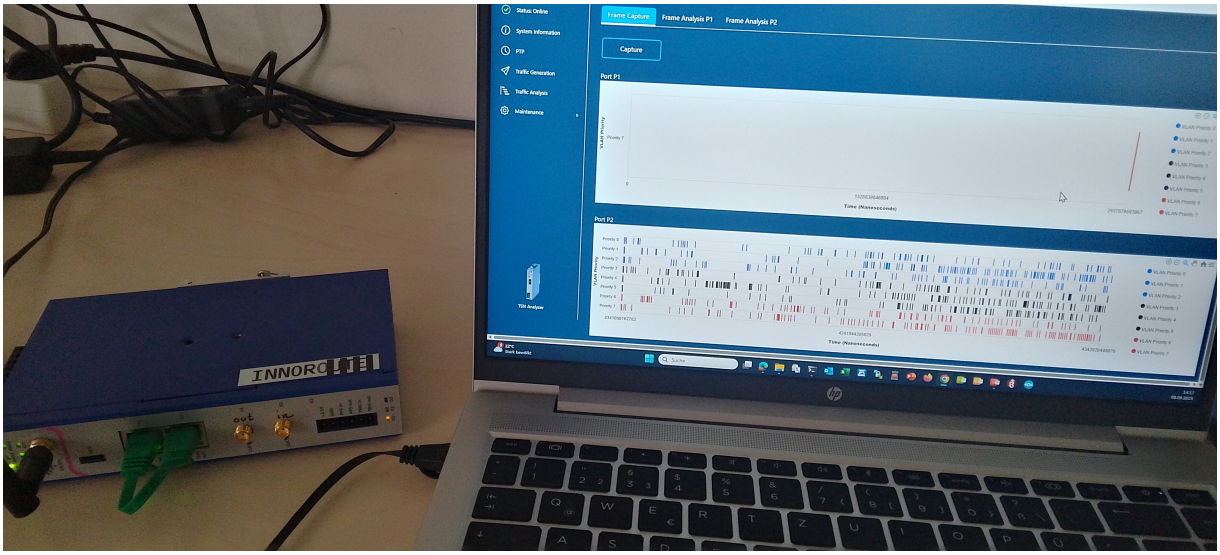

Figure 7: TSN-Analyzer self-measurement

Test of RTH Flow Cache and TAS - Use Case 1

For this test, the output port P1 of the TSN-Analyzer is connected with a Gray cable to to the left port (RT0) of the RTH and the right port (RT2) of the RTH is connected with a Yellow cable to the input port P2 of the TSN-Analyzer as shown in Figure 8.

The TSN-Analyzer still generates the eight streams shown in Figure 6, but none of them passes the RTH. According to Figure 2, as long as no entry is configured in the Flow Cache, all packets are forwarded to the Pi4. If the Pi4 is not configured as Router the packets are not forwarded and the Traffic Analysis screen at the right remains blank.

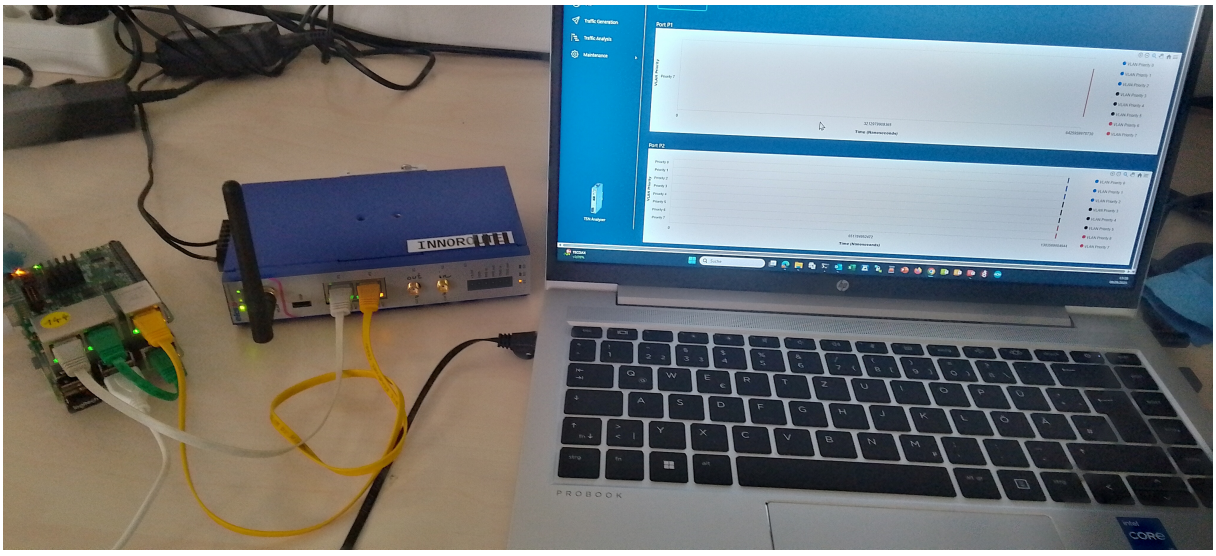

Figure 8: RTH Use Case 1 measurement with TSN-Analyzer

Next, the RTH Flow Cache is configured with four entries in such a way that three flows are forwarded to RT2 and one flow is reflected back to RT0:

VLAN-ID	PCP	Output	Queue
1	0	RT2	0

VLAN-ID	PCP	Output	Queue
4	3	RT2	3
5	4	RT2	4
8	7	RT0	7

The result is that these four out of eight generated flows (Figure 6) are forwarded directly to a RTH port as shown in Figure 9.

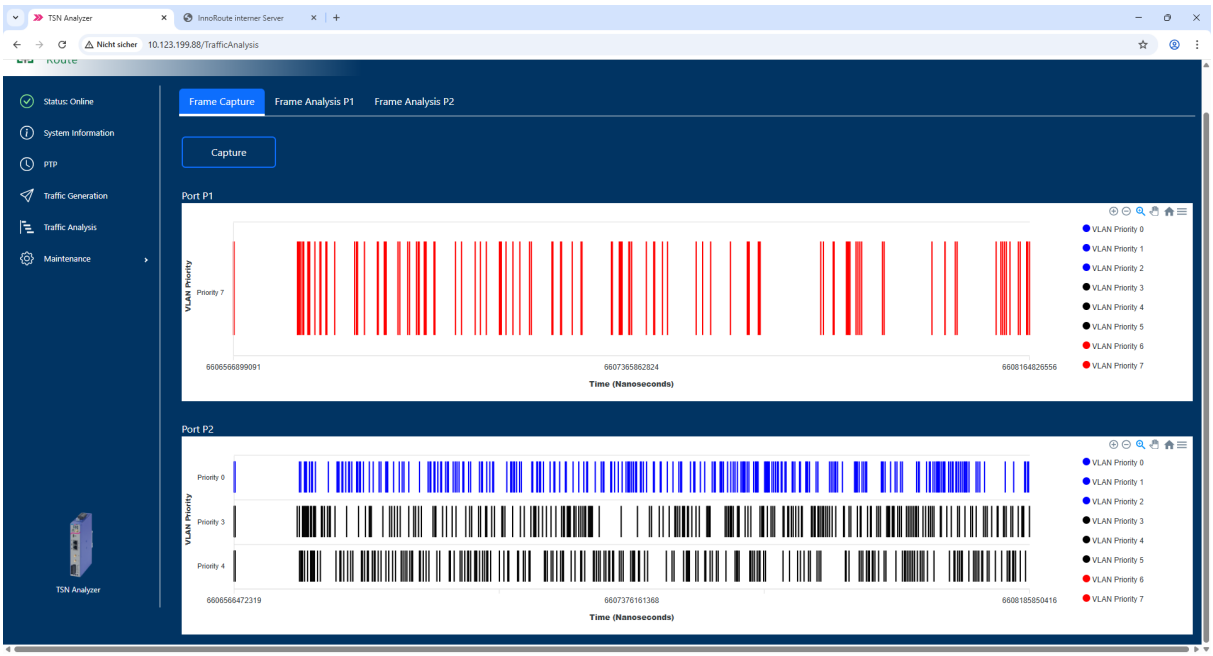

Figure 9: Snapshot of forwarded packet streams

In Figure 9, the TAS was not configured. Now, the shaping of Queue3 is configured using the Python Code from page 6. The result is shown in Figure 10. One can see that Priority 3 stream is regularly shaped, while the other streams are still stochastic.

Figure 10: Snapshot of forwarded packet streams with TAS-shaped PCP=3 stream

6 Use Case 4

This Use Case is a Gigabit TAP function according to the concept shown in Figure 11.

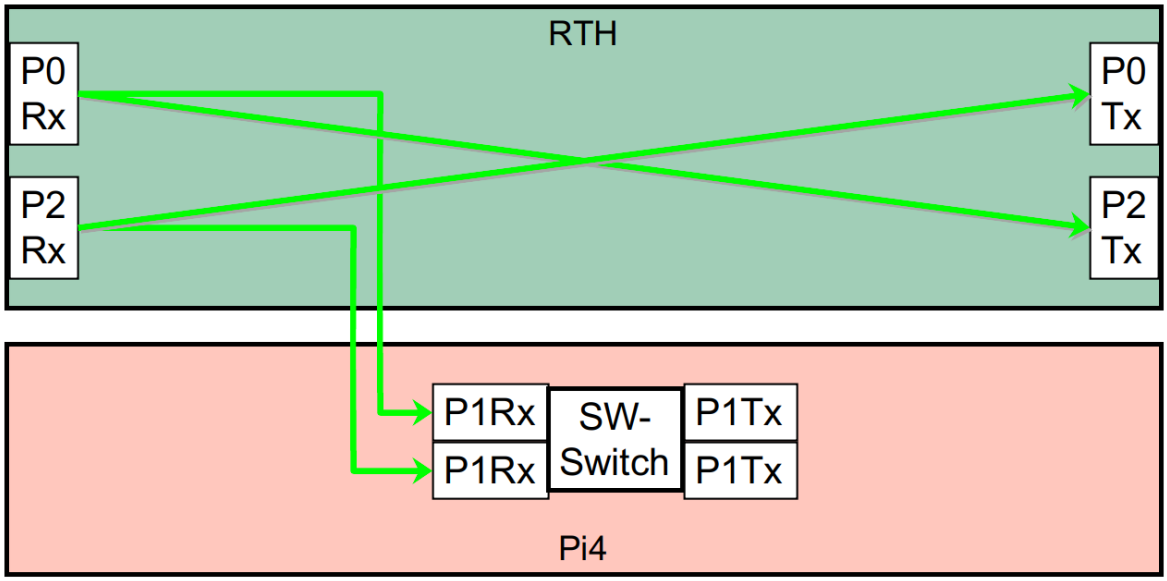

Figure 11: RTH packet flows for Use Case 4

Tbd.

7 Literature

All further information about Real-Time HAT operation will be published on https://github.com/InnoRoute/RealtimeHAT .